

Lab exercises

Part 03 — Natural Language Processing · 03-nlp/lab/EXERCISES.md

Lab — NLP & Transformers

1. Tokenizer archaeology (1.5h)

Install `tiktoken` and `transformers`. For the same 5 paragraphs of text, compare token counts across `cl100k_base`, `o200k_base`, a Llama tokenizer, and a multilingual one. Then repeat with: Python code, JSON, a table of numbers, Hindi, Chinese, and emoji.

Deliverable: a matrix of chars-per-token. **Checks:** English prose lands near 4 chars/token for modern vocabularies. Non-Latin scripts are dramatically worse on English-centric tokenizers. Explain in two sentences what this does to your API bill if you serve a multilingual product.

2. Reproduce the "strawberry" failure (1h)

Take a real tokenizer.

- **2a.** Tokenize `strawberry`, `raspberry`, `blueberry`. Print the decoded pieces.
- **2b.** Explain precisely why counting `r` is hard given those pieces.
- **2c.** Design a prompt that makes a model succeed anyway. (Hint: make the letters into tokens.)
- **2d.** Explain why 2c works in terms of what the model can actually attend to.

3. Implement attention yourself, no copying (3h)

From the equations in `../notes/02-transformers.md` — not from the code file — implement single-head attention, then multi-head, then a causal mask.

Checks: - Your attention weight rows sum to 1.0 (assert it). - With a causal mask, `np.triu(weights, 1).sum() == 0` exactly. - Your multi-head output shape matches the input shape. - **The real test:** compare against `torch.nn.functional.scaled_dot_product_attention` and match to 1e-5.

4. Ablate the transformer block (2h)

Using `../code/attention_from_scratch.py` as a base, measure activation std over 12 layers for each of: (a) Pre-LN, (b) Post-LN, (c) no LayerNorm, (d) no residual connections.

Deliverable: a 4-line plot or table. **Checks:** (c) explodes exponentially. (d) is worse than you expect. Explain why (a) beat (b) historically in one paragraph, and connect it to why warmup exists.

5. RoPE and context extension (2h)

- **5a.** Implement RoPE. Verify numerically that $q_m \cdot k_n$ depends only on $m - n$.
- **5b.** Implement position interpolation: divide all positions by a factor s . Show that a model trained for length L now maps length sL into the trained angular range.
- **5c.** Read the YaRN paper's abstract and write three sentences on why naive interpolation loses high-frequency detail.

6. KV cache calculator (1.5h)

Write a function `kv_cache_gb(layers, kv_heads, head_dim, seq_len, batch, dtype)`.

Checks: - Reproduce the numbers in demo 9 of the code file. - Answer: on an 80 GB GPU running a 70B GQA model in fp16 (weights $\approx$ 140 GB, so assume 2 GPUs and $\sim$ 20 GB free for cache), how many concurrent 32k-token sessions can you serve? Show your working. - Then answer the same for MHA instead of GQA, and state the ratio.

7. Fine-tune an encoder and beat an LLM on cost (3h)

Pick a text classification dataset (AG News, IMDb, or your own).

- **7a.** Prompt a frontier LLM zero-shot. Record accuracy, latency, and cost per 1000 items.
- **7b.** Fine-tune a small encoder (`distilbert-base`, `ModernBERT-base`) on the training split.
- **7c.** Compare on the same test set.

Check: the fine-tuned encoder should be within a few points of the LLM — often better — at roughly $100\times$ lower cost per item. Write the two-paragraph recommendation you would actually send to an engineering manager, including where the LLM still wins (cold start, no labels, changing label set).

8. Capstone — a working mini-GPT (4h)

Follow Karpathy's "Let's build GPT" and produce a character-level transformer trained on a text file you choose. Then modify it:

- swap learned positional embeddings for RoPE,
- swap Post-LN for Pre-LN,
- add a KV cache to generation and measure the speedup.

Check: generation with a KV cache is dramatically faster and produces **identical** output to generation without it. If the outputs differ, your cache is wrong — that is the single most instructive bug in this module.